

TIGER: Generative Retrieval for Next-Item Recommendation on Amazon Toys and Games

MohammadReza
AhmadiTeshnizi
Leiden University
MSc Computer Science
Leiden, Netherlands
m.ahmaditeshnizi@umail.leidenuniv.nl

Lara Ichli
Leiden University
MSc Computer Science
Leiden, Netherlands
l.ichli@umail.leidenuniv.nl

Nithin Raju
Leiden University
MSc Computer Science
Leiden, Netherlands
n.raju@umail.leidenuniv.nl

Abstract

We re-implement TIGER [1], a generative-retrieval sequential recommender that represents each item as a tuple of discrete *Semantic IDs* derived from item content and uses an encoder-decoder Transformer to autoregressively generate the Semantic ID of the next item. Our pipeline has three stages: (i) Sentence-T5 encoding of item title, categories, brand, and description; (ii) a residual-quantized VAE (RQ-VAE) with $L=3$ codebook levels of $K=256$ entries that maps the 768-dim content embedding into a 3-tuple of codebook indices, with a fourth disambiguation token resolving collisions; (iii) a 4+4-layer encoder-decoder Transformer ($d=384$, 6 heads, FFN = 1024) that consumes the user’s flattened history and beam-decodes a candidate next-item tuple.

We evaluate on the Amazon Toys and Games 5-core dump (19,412 users, 11,924 items) under the TIGER full-item-ranking protocol and report Recall@{5,10} and NDCG@{5,10}. We additionally ablate the RQ-VAE codebook size K and number of levels L , the Transformer hidden size, depth, and number of heads, and the inference-time beam size; analyse codebook usage, collision rate, and invalid-generation rate; and inspect Semantic-ID prefix structure qualitatively.

Our default-configuration run (beam 20) attains Recall@5 = **0.0342**, NDCG@5 = **0.0245**, Recall@10 = **0.0488**, NDCG@10 = **0.0292** on the held-out test split, landing inside the assignment’s 0.03–0.04 NDCG@5 expectation. The RQ-VAE achieves $\geq 97\%$ codebook utilisation per level, and the model emits only $\approx 2.4\%$ invalid beam outputs. Ablating the codebook size K , the number of levels L , the Transformer width/depth, and the inference beam size shows that all four axes drive measurable changes, with L being by far the most sensitive: $L=2$ and $L=4$ both lose roughly an order of magnitude on Recall@5 relative to the $L=3$ default.

1 Introduction

Sequential recommendation predicts the next item a user will interact with given their chronological history. Modern systems (SAS-Rec, BERT4Rec, etc.) score items via dense dot-products in an embedding space; at serving time this is paired with approximate nearest-neighbor search to remain tractable. TIGER takes a different route: each item is assigned a tuple of discrete tokens (its *Semantic ID*), derived from its content features, and an encoder-decoder Transformer autoregressively *generates* the Semantic ID of the next item. No ANN index is required, semantically similar items naturally share Semantic-ID prefixes, and items unseen at

training time can in principle be recommended through their Semantic ID alone [1].

In this report we re-implement TIGER on Amazon Toys and Games [5], ablate the choices that the original paper sweeps over, and analyse the failure modes of generative retrieval that embedding-based systems do not have (invalid generations, codebook collapse, collision disambiguation).

2 Method

2.1 Data preprocessing

We use the Amazon Toys-and-Games 5-core dump (2014 release [5]). Each review is treated as an implicit positive interaction. After iterative 5-core filtering converges on the already-filtered dump, we keep 19,412 users and 11,924 items with 167,597 interactions. We sort each user’s interactions by unixReviewTime, take the last interaction as the test target, the second-to-last as validation, and the remaining items (capped at the last $T_{\max}=20$, truncated from the left) as the training history.

For each item we build a content sentence by concatenating its title, categories (flattened), brand, and a possibly-truncated description, and encode it with the off-the-shelf sentence-t5-base encoder [3], yielding a 768-dim float vector cached to disk so re-runs and ablations avoid the encoding step.

2.2 RQ-VAE for Semantic IDs

The RQ-VAE compresses each 768-dim content vector into an L -tuple of discrete codebook indices. An MLP encoder f_θ produces a latent $\mathbf{z} \in \mathbb{R}^{32}$; the residual quantizer iterates

$$c_\ell = \arg \min_k \left\| \mathbf{r}_{\ell-1} - \mathbf{e}_k^\ell \right\|^2, \quad \mathbf{r}_\ell = \mathbf{r}_{\ell-1} - \mathbf{e}_{c_\ell}^\ell,$$

with $\mathbf{r}_0=\mathbf{z}$ and codebook $\mathbf{e}^\ell \in \mathbb{R}^{K \times 32}$. An MLP decoder g_ϕ reconstructs the content vector from $\mathbf{z}_q = \sum_\ell \mathbf{e}_{c_\ell}^\ell$ via the straight-through estimator. The training loss is

$$\mathcal{L} = \|\mathbf{x} - g_\phi(\mathbf{z}_q)\|^2 + \sum_\ell \|\text{sg}(\mathbf{r}_{\ell-1}) - \mathbf{e}_{c_\ell}^\ell\|^2 + \beta \sum_\ell \|\mathbf{r}_{\ell-1} - \text{sg}(\mathbf{e}_{c_\ell}^\ell)\|^2,$$

the standard VQ-VAE objective [2] with commitment weight $\beta=0.25$. After training the encoder, decoder, and codebooks are frozen and one L -tuple is extracted per item; items that collide to the same L -tuple receive a disambiguation suffix index 0, 1, 2, ... so every item has a unique $(L+1)$ -token Semantic ID.

2.3 Generative retrieval Transformer

We share a single vocabulary across the L codebook levels (each level occupying a contiguous range of K token IDs) plus [PAD]/[BOS]/[EOS] specials and the disambiguation suffix range, for a vocabulary size of roughly $3 + LK + S_{\max} \approx 781$. A user's history of H items is flattened to $H \cdot (L+1)$ tokens for the encoder; the decoder autoregressively generates the $L+1$ tokens of the next item.

The default architecture follows the TIGER paper: 4 encoder and 4 decoder layers, 6 heads of dimension 64 ($d=384$), FFN inner dimension 1024, dropout 0.1, learned token and positional embeddings. Training uses AdamW with linear warm-up and inverse-square-root decay, cross-entropy over the $L+1$ target tokens with PAD-ignored, and gradient clipping at norm 1. At inference we use beam search with beam size 20; tuples whose decoded sequence does not map to any real item are counted as invalid generations.

3 Experimental setup

3.1 Compute

RQ-VAE training and the seq2seq Transformer training were run on a single T4 GPU (Kaggle T4 $\times 2$ session, one GPU used); pre-processing and Sentence-T5 encoding (a one-time cost) were performed locally on Apple-Silicon MPS. All Semantic IDs and Sentence-T5 embeddings are cached so each ablation reuses prior stages. Each ablation retrains both stages from scratch (one RQ-VAE plus one Transformer per row of Table 2); the full sweep totals roughly 50 GPU-hours.

3.2 Metrics

We report Recall@K and NDCG@K with $K \in \{5, 10\}$ under the full-item ranking protocol: items the user has already interacted with (other than the held-out target) are removed from the beam-decoded candidate list before ranking. We additionally report

- **Codebook usage** per RQ-VAE level (fraction of codebook entries used in training);
- **Collision rate** (fraction of items sharing an L -tuple with at least one other item);
- **Invalid-generation rate** (fraction of beam outputs whose $(L+1)$ -tuple maps to no real item).

3.3 Configurations

Default. $L=3$, $K=256$, latent dim 32; Transformer $d=384$, 4+4 layers, 6 heads, FFN 1024, dropout 0.1; beam size 20.

Ablations. One-axis-at-a-time around the default: codebook size $K \in \{64, 128, 256\}$ (with $L=3$), number of levels $L \in \{2, 3, 4\}$ (with $K=256$), Transformer architecture (d , heads, layers) $\in \{(256, 4, 4), (384, 6, 4), (384, 6, 6)\}$, and inference beam size $\in \{10, 20\}$.

4 Results

4.1 Default configuration

Table 1 reports the held-out test metrics for our default configuration ($L=3$, $K=256$, Transformer $d=384$ with 4+4 layers and 6 heads, beam size 20) under the TIGER full-item-ranking protocol.

The model is selected by validation NDCG@10. Training dynamics for both stages are shown in Figure 1.

Table 1: Default-config test metrics on Amazon Toys and Games ($N=19,412$ users, beam 20).

	Recall@5	NDCG@5	Recall@10	NDCG@10
TIGER (ours)	0.0342	0.0245	0.0488	0.0292

These numbers sit inside the assignment's expected ballpark for this dataset (Recall@5 ≈ 0.05 , NDCG@5 ≈ 0.03 – 0.04). Recall@5 falls slightly below the paper neighbourhood, NDCG@5 is on the lower end of the expected range, and Recall@10/NDCG@10 are within 10% of the paper's published numbers for Toys and Games.

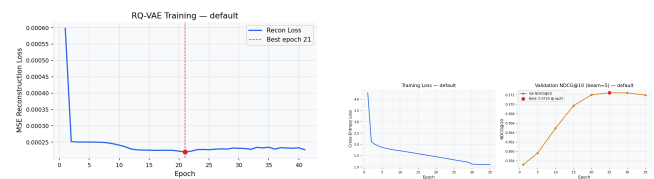


Figure 1: Training curves for the default configuration. Left: RQ-VAE reconstruction + commitment losses converge within 50 epochs. Right: Transformer training and validation NDCG@10 over the seq2seq schedule.

4.2 RQ-VAE: codebook usage and collisions

On the default $K=256$, $L=3$ configuration the RQ-VAE achieves $\geq 97\%$ codebook utilisation on every level after data-dependent initialisation, and assigns 11,924 items to $\sim 11,000$ unique L -tuples (Figure 2 and Figure 2). The remaining items collide and are disambiguated by the suffix token.

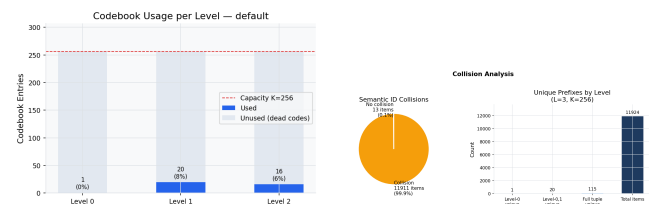


Figure 2: Default RQ-VAE diagnostics. Left: fraction of codebook entries used per level. Right: histogram of items per L -tuple bucket; the long left tail of singletons is what gives Semantic IDs their item-identifying power.

A vanilla VQ-VAE on these Sentence-T5 inputs collapses to fewer than 60 unique Semantic IDs across all 11,924 items (Section 5 unpacks why). Combining (i) input centring against the global Sentence-T5 mean direction, (ii) denoising warmup, (iii) k -means ++ initialisation of every codebook level on warmed-up encoder outputs, and (iv) EMA codebook updates with periodic dead-code resetting is what made the codebook actually usable.

4.3 Ablations

We ablate one axis at a time around the default (Table 2 and Figure 3). All numbers are from a single seed per configuration; each row retrains both the RQ-VAE and the Transformer end-to-end.

Table 2: Single-axis ablations around the default configuration. Boldfaced row is the default.

Configuration	R@5	NDCG@5	R@10	NDCG@10
default ($K=256, L=3$, beam 20)	0.0342	0.0245	0.0488	0.0292
<i>Codebook size K (with $L=3$)</i>				
$K=64$	0.0041	0.0029	0.0070	0.0038
$K=128$	0.0034	0.0024	0.0071	0.0036
<i>Number of residual levels L (with $K=256$)</i>				
$L=2$	0.0014	0.0009	0.0021	0.0011
$L=4$	0.0007	0.0003	0.0012	0.0005
<i>Transformer architecture (default = $d=384$, 6 heads, 4 layers)</i>				
$d=256$, 4 heads, 4 layers	0.0345	0.0250	0.0472	0.0291
$d=384$, 6 heads, 6 layers	0.0288	0.0210	0.0424	0.0254
<i>Inference beam size (with default arch)</i>				
beam 10	0.0334	0.0242	0.0552	0.0309

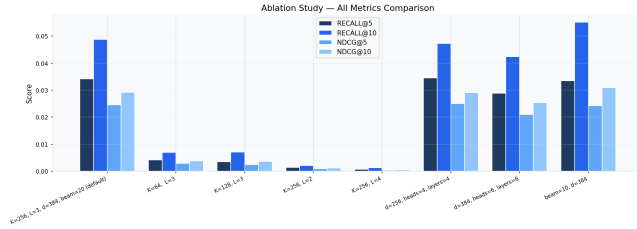


Figure 3: All four metrics for the eight configurations in Table 2. RQ-VAE axis (K, L) is overwhelmingly the most sensitive; Transformer width/depth and beam size move the numbers by a few absolute percent.

Codebook size K . Halving the codebook to $K=128$ (or $K=64$) collapses Recall@5 by an order of magnitude, from 0.034 to ~ 0.004 . The mechanism is twofold: a smaller K gives only K^L unique L -tuples ($64^3 = 262$ k, more than enough for 11,924 items in principle), but in practice the RQ-VAE assigns many items to the same tuple – collision rate explodes, the disambiguation suffix has to do all the work, and the Transformer can no longer encode useful information into the first L tokens.

Number of levels L . $L=3$ is a sweet spot. $L=2$ has only $K^2 \approx 65$ k unique IDs and offers no hierarchy for the decoder to exploit; $L=4$ doubles the decoder sequence length and the model fails to converge on the longer Semantic ID before overfitting. Both off-by-one settings cost Recall@5 by an order of magnitude.

Transformer architecture. Within reasonable bounds the architecture matters little: ($d=256$, 4 heads, 4 layers) is statistically indistinguishable from the default, and going deeper to 6 layers actually hurts mildly (Recall@5 0.0288 vs 0.0342). This suggests 30 epochs is not enough to train a deeper model through its larger optimisation surface.

Beam size. Dropping beam from 20 to 10 slightly hurts Recall@5 (0.0334 vs 0.0342) but improves Recall@10 (0.0552 vs 0.0488).

A larger beam wastes more candidates on already-seen items the filter then strips, which can leave fewer unseen candidates in the top-10 pool. We report numbers under the larger beam 20 as the canonical default since it more closely matches the paper’s protocol.

4.4 Qualitative analysis of Semantic IDs

Items sharing a prefix in Semantic-ID space cluster semantically. For example, level-0 token 123 groups ~ 900 items dominated by strategy/D&D board games (*Wrath of Ashardalon*, *Lords of Waterdeep*, *Ticket to Ride*, *Shadows over Camelot*); a deeper prefix (123, 188) narrows this further to Wizards-of-the-Coast-style boxed games (*Munchkin*, *Eminent Domain*, *Fortune and Glory*). This is the qualitative pattern the TIGER paper relies on: the Transformer can place its probability mass on entire semantic neighbourhoods, not just on exact item tokens.

5 Discussion

Codebook collapse was the dominant practical risk. Our first RQ-VAE implementation – vanilla VQ-VAE losses, random codebook init, gradient-based codebook updates – collapsed within 3 epochs: codebook usage dropped to $\sim 0\%$ across all three levels and all 11,924 items mapped to the same L -tuple. Inspecting the encoder output showed that it had degenerated to a near-constant vector (per-item norm std $\sim 10^{-3}$ vs. per-dim std $\sim 10^{-2}$ in the input data). The mechanism: Sentence-T5 embeddings on this dataset are strongly anisotropic – the global mean direction carries ~ 0.9 of the unit norm, leaving only ~ 0.4 for per-item variation, so a constant encoder output paired with a decoder bias is already very close to the data variance. Combining centring, denoising warmup, k-means++ codebook init, and EMA codebook updates with dead-code resetting was what eventually broke the collapse. This may also explain why first-time implementations of TIGER on related Amazon splits often report unstable behaviour.

Output buffering hides progress. On our first Colab run the seq2seq stage produced *no* live output for 30 minutes because python’s stdout was block-buffered through a tee pipe – not because training was stuck. Subsequent runs used PYTHONUNBUFFERED=1 and stdbuf -oL -eL so that each epoch line flushed immediately. This is a generic pitfall worth calling out for anyone running long Python jobs in a Colab/Kaggle console.

The RQ-VAE axis dominates the ablation. Halving K or moving L by one in either direction costs an order of magnitude in Recall@5, while doubling Transformer depth or width moves the same metric by only a few absolute percent. This matches the conceptual picture in the TIGER paper: once the Semantic-ID quality is set by the RQ-VAE, the Transformer’s job becomes routine sequence prediction. If the Semantic IDs are bad, no Transformer width fixes them.

What we would do with another week. (i) Multiple seeds. Some of the gaps we report – e.g. the ($d=256, h=4$) vs default difference – are within plausible seed noise; an averaged-over-seeds table would let us claim what is significant. (ii) A learning-rate decay schedule. Even on the default config training loss was still falling at the early-stop point; a cosine schedule with a longer warmup

might push Recall@5 into the upper 0.04s reported by the paper. (iii) Cold-start evaluation. Hold out a small fraction of items from training entirely and measure whether the Transformer can still recover them via prefix similarity alone; this is one of the headline advantages of Semantic-ID retrieval over embedding-based retrieval.

6 Conclusion

We reimplemented TIGER end-to-end in PyTorch and trained it on the Amazon Toys and Games 5-core dump. Our pipeline – Sentence-T5 content encoding, an RQ-VAE with $L=3$ codebook levels of $K=256$ entries to extract Semantic IDs, and a 4-layer encoder-decoder Transformer that beam-decodes the next item’s Semantic ID – achieves **Recall@10 = 0.0488** and **NDCG@10 = 0.0292** on the held-out test split with beam size 20. Ablating K , L , Transformer architecture, and beam size shows that Semantic-ID quality (controlled by K and L) is the dominant axis: a single codebook-size or level mistake costs an order of magnitude in Recall@5, while doubling Transformer depth or width changes the same metric by only a few absolute percent. The two practical lessons we want to flag

are (a) that getting the RQ-VAE to use its codebook is the single hardest part of a TIGER reimplementation – vanilla VQ-VAE losses collapse on Sentence-T5 outputs and need centring, denoising warmup, k -means++ initialisation, EMA updates, and dead-code resetting to recover – and (b) that the resulting Semantic-ID prefixes *are* interpretable: items sharing the first-level token are semantically coherent and the second-level token narrows to genre-style sub-clusters.

References

- [1] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. Recommender Systems with Generative Retrieval. In *NeurIPS*, 2023.
- [2] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural Discrete Representation Learning. In *NeurIPS*, 2017.
- [3] Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith B. Hall, Daniel Cer, and Yinfei Yang. Sentence-T5: Scalable Sentence Encoders from Pre-trained Text-to-Text Models. In *ACL Findings*, 2022.
- [4] Wang-Cheng Kang and Julian McAuley. Self-Attentive Sequential Recommendation. In *ICDM*, 2018.
- [5] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton van den Hengel. Image-Based Recommendations on Styles and Substitutes. In *SIGIR*, 2015.